Algorithm 2 FLAG-TRADER with PPO

Input: Pre-trained LLM parameters $(\theta_{\texttt{frozen}}, \theta_{\texttt{train}})$; actor parameters θ_P; critic parameters θ_V; environment $\mathcal{E}$; discount factor γ; GAE parameter λ; PPO clip ε; learning rates η, β;
Initialize $\theta_{\text{train}}, \theta_P, \theta_V$; let $\theta_{\text{old}} \leftarrow \theta$
Initialize replay buffer $B \leftarrow \emptyset$
for iteration = 1 to max_iters **do**
  // Collect Rollouts
  for t = 1 to T **do**
    Fetch the current state s_t from the environment and construct an input prompt $\texttt{lang}(s_t)$;
    Pass prompt $\texttt{lang}(s_t)$ through LLM;
    POLICY_NET outputs a_t from action space {"`buy`," "`sell`," "`hold`"} based on (8);
    Execute action a_t in the environment and observe reward $r(s_t, a_t)$ and transition to new state s_{t+1};
    Store experience tuple (s_t, a_t, r_t, s_{t+1}) in replay buffer B;
  end for
  // Compute Advantage and Targets
  for each transition in B **do**
    Compute $V_{\theta_{value}}(s_t)$ and advantage A_t (e.g., via GAE)
  end for
  // Perform PPO Updates
  for update_epoch = 1 to K **do**
    Sample mini-batch $\mathcal{M}$ from B
    Compute probability ratio $r_t(\theta_{policy}) = \frac{\pi_{\theta_{policy}}(a_t|s_t)}{\pi_{\theta_{policy,\text{old}}}(a_t|s_t)}$;
    Compute PPO loss $\mathcal{L}_P(\theta_{policy}) = \mathbb{E}_t\Big[\min\big(r_t(\theta_{policy})\,A_t,\ \text{clip}\big(r_t(\theta_{policy}),\,1-\varepsilon,\,1+\varepsilon\big)\,A_t\big)\Big]$;
    Compute Value loss $\mathcal{L}_V(\theta_{value}) = \mathbb{E}_t\Big[(V_{\theta_{value}}(s_t) - R_t)^2\Big]$;
    Compute total loss $\mathcal{L}_{\text{total}}(\theta) = -\mathcal{L}_P(\theta_{policy}) + c_1\,\mathcal{L}_V(\theta_{value}) - c_2\,\mathcal{H}\big(\pi_{\theta_{policy}}\big)$;
    Perform gradient descent on each parameter group:

$$\theta_P \leftarrow \theta_P - \eta\,\nabla_{\theta_P}\mathcal{L}_P,$$
$$\theta_V \leftarrow \theta_V - \eta\,\nabla_{\theta_V}\mathcal{L}_V,$$
$$\theta_{\texttt{train}} \leftarrow \theta_{\texttt{train}} - \beta\,\nabla_{\theta_{\text{train}}}\mathcal{L}_{\text{total}};$$

  end for
  // Update old policy parameters
  Update $\theta = \big(\theta_{\texttt{train}}, \theta_P, \theta_V\big)$ by $\theta_{\text{old}} \leftarrow \theta$;
end for
Return: Fine-tuned POLICY_NET(θ_P).

B Additional Experimental Details

Hyperparameters for Finetuening FLAG-TRADER with PPO in Algorithm 2

Table 3: FLAG-TRADER with PPO Finetuning Hyperparameters and Settings.

Parameter	**Default Value**	**Description**
total_timesteps	13860	Total number of timesteps
learning_rate	5×10^{-4}	Learning rate of optimizer
num_envs	1	Number of parallel environments
num_steps	40	Steps per policy rollout
anneal_lr	True	Enable learning rate annealing
gamma	0.95	Discount factor γ
gae_lambda	0.98	Lambda for Generalized Advantage Estimation
update_epochs	1	Number of update epochs per cycle
norm_adv	True	Advantages whitening
clip_coef	0.2	Surrogate clipping coefficient
clip_vloss	True	Clipped loss for value function
ent_coef	0.05	Coefficient of entropy term
vf_coef	0.5	Coefficient of value function
kl_coef	0.05	KL divergence with reference model
max_grad_norm	0.5	Maximum gradient clipping norm
target_kl	None	Target KL divergence threshold
dropout	0.0	Dropout rate
llm	"SmolLM2-135M-Instruct"	Model to fine-tune
train_dtype	"float16"	Training data type
gradient_accumulation_steps	8	Number of gradient accumulation steps
minibatch_size	32	Mini-batch size for fine-tuning
max_episode_steps	65	Maximum number of steps per episode